Лабораторная работа №8: Событийно-ориентированная подсистема мониторинга и валидации: Observer и Chain of Responsibility

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Неделя: 8 из 10
Формат: Лабораторная работа, 3 академических часа (135 минут)
Среда выполнения: Python 3.12+, pytest, Git, VS Code / PyCharm

---

1. Цель и контекст работы

Цель работы: Применить поведенческие паттерны проектирования GoF (**Chain of Responsibility** и **Observer**) для построения расширяемого конвейера проверки платежей и слабой связи подсистем мониторинга в валидаторе Avtobys.
В реальной эксплуатации: 1. Проверка права на проезд представляет собой многоступенчатый фильтр: проверка целостности RFID-чипа → проверка стоп-листа → защита от двойного прикладывания (Double Tap) → проверка баланса → авторизация списания. Использование паттерна **Chain of Responsibility** позволяет добавлять новые правила проверки в конвейер (например, проверка зонального разрешения или студенческого билета) без модификации остальных узлов. 2. При совершении транзакции независимые системы должны быть мгновенно оповещены: локальный дисплей, счетчик метрик Prometheus и диспетчерская служба безопасности. Это реализуется через паттерн **Observer**.

---

2. Структура репозитория и требования к сдаче

Работа выполняется в папке lab08_behavioral_pipeline/:

```
lab08_behavioral_pipeline/
├── chain/
│   ├── __init__.py
│   ├── base.py          # Базовый абстрактный обработчик ValidationHandler
│   └── handlers.py      # Узлы цепочки: Blacklist, Cooldown, Balance, Executor
├── observer/
│   ├── __init__.py
│   ├── subject.py       # Издатель событий TransitTelemetrySubject
│   └── observers.py     # Наблюдатели: Metrics, SecurityAlert, FileAudit
├── pipeline/
│   ├── __init__.py
│   └── engine.py        # Объединенный движок валидации и телеметрии
├── tests/
│   ├── __init__.py
│   ├── test_chain.py    # Тесты прерывания и прохождения цепочки
```

```
│   └── test_observer.py  # Тесты подписки и изоляции наблюдателей
└── README.md             # UML диаграмма последовательности (Sequence Diagram)
```

Критерии оценки (20 баллов):

Блок	Задание	Баллы
Блок 1	Реализация конвейера Chain of Responsibility с 4 узлами	6
Блок 2	Разработка издателя Subject и 3 независимых наблюдателей Observer	6
Блок 3	Интеграция цепочки и наблюдателей в единый валидационный движок	4
Блок 4	Тестирование юнит-тестами pytest с проверкой граничных условий	4

---

3. Практические задания

Блок 1. Конвейер Chain of Responsibility (6 баллов)

В файле chain/base.py определите базовый класс:

```python
class ValidationHandler(ABC):
    def __init__(self):
        self._next_handler: ValidationHandler | None = None

    def set_next(self, handler: "ValidationHandler") -> "ValidationHandler":
        self._next_handler = handler
        return handler

    @abstractmethod
    def handle(self, request: PaymentRequest) -> ValidationResult:
        if self._next_handler:
            return self._next_handler.handle(request)
        return ValidationResult(is_approved=True)
```

В chain/handlers.py реализуйте: 1. BlacklistCheckHandler: если card_id в стоп-листе → прерывает цепочку с ValidationResult(is_approved=False, reason="Карта в черном списке"). 2. AntiFraudCooldownHandler: если та же карта приложена повторно менее чем через 60 секунд на том же валидаторе → прерывает цепочку с ошибкой. 3. BalanceCheckHandler: если баланс карты меньше стоимости проезда → прерывает цепочку с ошибкой нехватки средств. 4. TransactionExecutionHandler: финальный узел, списывает баланс и возвращает успешный результат с номером транзакции.

Блок 2. Подсистема оповещения Observer (6 баллов)

В файле observer/subject.py: - Реализуйте класс TransitTelemetrySubject с методами attach(), detach() и notify(event: TelemetryEvent).

В файле observer/observers.py: - LocalAuditObserver: записывает успешные и отклоненные транзакции в локальный лог. - MetricsCollectorObserver: увеличивает счетчики total_transactions, approved_count, rejected_count. - SecurityAlertObserver: если транзакция отклонена по причине фрода или стоп-листа, отправляет срочный алерт (выводит предупреждение в консоль безопасности).

Блок 3. Интеграция в движок (4 балла)

В pipeline/engine.py создайте фасадный класс TransitValidationPipeline, который: 1. Прогоняет входящий PaymentRequest через цепочку обязанностей. 2. При любом исходе (успех или отказ) публикует событие TelemetryEvent через TransitTelemetrySubject. 3. Возвращает клиенту итоговый статус.

Блок 4. Автоматическое тестирование (pytest) (4 балла)

В tests/: 1. Проверьте прерывание цепочки: если карта в стоп-листе, узлы BalanceCheck и Execution вообще не должны вызываться. 2. Проверьте наблюдателей: при отклонении транзакции SecurityAlertObserver вызывается, а счетчик rejected_count в MetricsCollector увеличивается на 1. 3. Проверьте отписку: метод detach() гарантирует, что отписавшийся наблюдатель больше не получает уведомлений.
pytest tests/ -v

---

4. Контрольные вопросы для защиты работы

1. Чем отличается поведение цепочки обязанностей, прерывающейся на первой ошибке (Short-circuiting), от цепочки, накапливающей все ошибки валидации?
2. Почему издатель событий (Subject) не должен содержать бизнес-логику самих наблюдателей?
3. Как динамическое построение цепочки позволяет на лету менять правила валидации для разных типов рейсов (городской vs междугородний)?
4. Каким образом паттерны Observer и Chain of Responsibility помогают соблюдать принцип Open/Closed (OCP)?